

MODUL PEMBELAJARAN
PENGEMBANGAN DASHBOARD IoT

Disusun untuk memenuhi Tugas Mata Kuliah Internet of Things

Dosen Pengampu:

Susilawati, S.Si., M.Si.



oleh:

Salina Putri Herawati (2310631170048)

Devi Rahmawati Agustin (2310631170072)

Jonathan Aditya Saputra (2310631170138)

Naufal Mirza Aldilla (2310631170144)

PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG
KARAWANG

2026

TUJUAN PEMBELAJARAN

Setelah mengikuti modul ini, mahasiswa mampu:

1. Memahami konsep dan fungsi dashboard IoT dalam sistem monitoring secara mendalam
2. Menjelaskan alur data end-to-end dari sensor hingga tampilan dashboard
3. Memahami konsep dan cara kerja web dashboard secara teknis
4. Mengimplementasikan Node-RED/Firebase/ThingsBoard sebagai teknologi pendukung
5. Merancang dan membuat visualisasi data IoT yang efektif
6. Memahami mekanisme monitoring melalui perangkat mobile
7. Menganalisis manfaat, kendala, dan solusi implementasi dashboard IoT secara kritis

BAB 1 – Pengantar Dashboard IoT

1.1. Pengertian IoT

Internet of Things (IoT) adalah paradigma di mana objek fisik sehari-hari dilengkapi dengan sensor, prosesor kecil, dan modul komunikasi, sehingga mampu mengumpulkan data dari lingkungannya dan mengirimkannya melalui jaringan tanpa campur tangan manusia secara langsung pada setiap prosesnya.

Tiga elemen inti IoT:

1. Sensing, kemampuan menangkap kondisi fisik (suhu, cahaya, gerak, berat, dll.)
2. Connectivity, kemampuan mengirim data melalui jaringan (Wi-Fi, Bluetooth, LoRa, seluler)
3. Actuation/Processing, kemampuan memproses data dan (opsional) melakukan tindakan balik

1.2. Pengertian Dashboard IoT

Dashboard IoT adalah lapisan antarmuka (interface layer) yang menjembatani antara data teknis mentah dari perangkat IoT dengan kebutuhan informasi pengguna akhir. Dashboard melakukan:

- Fetching/subscribing data dari database atau broker pesan
- Rendering data ke dalam elemen visual (chart, gauge, tabel)
- Refresh otomatis ketika ada data baru, tanpa interaksi pengguna

1.3. Fungsi Dashboard dalam Sistem IoT

Fungsi	Penjelasan
Menampilkan kondisi terkini	Menampilkan snapshot nilai sensor paling baru
Riwayat data (historical)	Menampilkan data sebelumnya untuk analisis tren dan evaluasi performa
Peringatan (alert)	Memberikan notifikasi otomatis saat nilai melewati ambang batas
Pusat kendali	Mengirim perintah dari dashboard ke aktuator/perangkat IoT
Pendukung keputusan	Membantu mempercepat analisis dan pengambilan keputusan

1.4. Perbedaan Dashboard Biasa dan Dashboard IoT

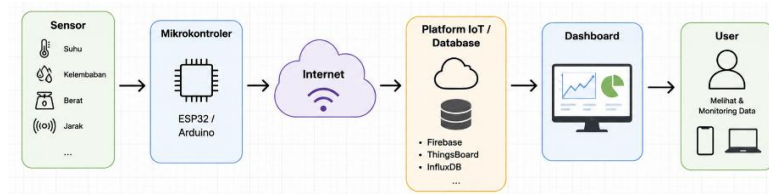
Aspek	Dashboard Biasa	Dashboard IoT
Sumber data	Input manual, file Excel/CSV	Sensor fisik secara otomatis
Frekuensi update	Manual/terjadwal	Otomatis, dapat per detik/milidetik
Mekanisme update	Refresh manual	Event-driven (listener) atau polling
Sifat data	Relatif statis dalam periode tertentu	Sangat dinamis dan terus berubah
Contoh	Dashboard penjualan bulanan	Dashboard suhu server, level tangki

1.5. Konsep Real-Time Monitoring

- Hard real-time = respons dalam batas waktu sangat ketat (sistem kendali kritis)
- Soft real-time = keterlambatan kecil masih ditoleransi (umum di dashboard IoT)
- Near real-time = istilah paling akurat untuk sebagian besar dashboard IoT web, karena hampir selalu ada delay kecil di sepanjang alur sistem

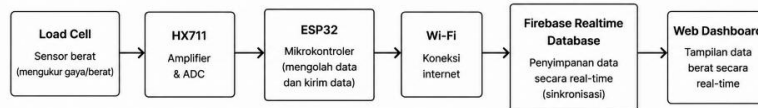
BAB 2 – Arsitektur Sistem Dashboard IoT

2.1. Alur Umum Sistem

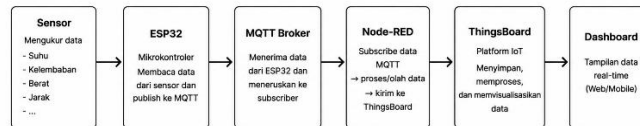


2.2. Contoh Implementasi

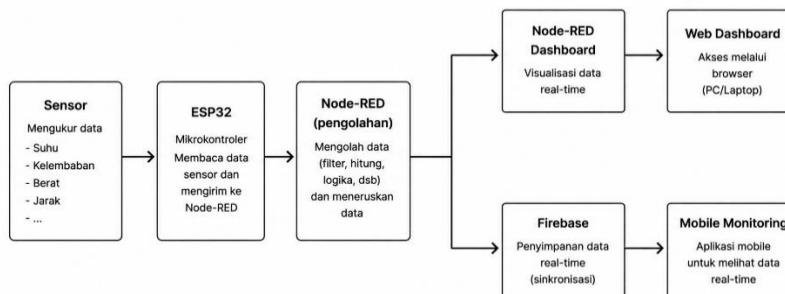
1. Skema A - Berbasis Cloud Database (Firebase)



2. Skema B - Berbasis Platform IoT



3. Skema C — Percabangan dari Satu Sumber



2.3. Komponen Sistem

Komponen	Peran
Sensor	Menangkap besaran fisik, seperti suhu, berat, jarak, dan kelembapan (DHT22, load cell, HC-SR04, dll.)
ESP32/Arduino	Membaca sinyal sensor dan mengubahnya menjadi data digital

Protokol komunikasi	Mengirimkan data antarperangkat menggunakan HTTP, MQTT, WebSocket, atau CoAP
Database/Cloud	Menyimpan data IoT, seperti Firebase, Firestore, atau InfluxDB
Backend	Melakukan validasi, kalkulasi, pemrosesan, dan routing data
Dashboard	Menampilkan data dalam bentuk visual yang mudah dipahami pengguna
User/Device	Menerima informasi dan dapat memberikan input atau perintah balik ke sistem

2.4. Perbandingan Skema

Aspek	Skema A (Firebase)	Skema B (MQTT + Node-RED + ThingsBoard)	Skema C (Percabangan)
Kompleksitas setup	Rendah–sedang	Sedang–tinggi	Sedang
Fleksibilitas	Tinggi	Sedang	Tinggi (2 kanal independen)
Skalabilitas	Cukup baik	Sangat baik	Baik
Cocok untuk pemula	Ya	Perlu pemahaman lebih	Ya, jika sudah paham Node-RED

BAB 3 – Data Visualization

3.1. Mengapa Data IoT Perlu Divisualisasikan

Sensor IoT bisa menghasilkan ribuan baris data per hari, sehingga tidak mungkin dibaca satu per satu dalam bentuk mentah. Visualisasi membantu mempercepat pemahaman kondisi sistem dan membuat pola atau tren misalnya kenaikan bertahap atau anomali mendadak, menjadi lebih mudah terlihat dibanding sekadar tabel angka. Dalam situasi kritis, visual yang mencolok (warna merah, alarm) juga mempercepat respons pengguna dibanding harus membaca log teks satu per satu.

3.2. Jenis-Jenis Visualisasi

Jenis	Kegunaan	Contoh Kasus
Card/KPI	Menampilkan nilai penting secara ringkas	“Suhu Saat Ini: 28,5°C”
Gauge	Menampilkan nilai dalam rentang tertentu	Level baterai 0–100%
Line Chart	Menampilkan tren dari waktu ke waktu	Grafik suhu selama 24 jam
Bar Chart	Membandingkan nilai antar kategori	Konsumsi listrik per jam
Table	Menampilkan data rinci atau log historis	Log sensor per menit
Status Indicator	Menampilkan status menggunakan warna atau ikon	Normal, Alert

3.3. Real-Time vs Historis

Visualisasi real-time (card, gauge, status indicator) menjawab pertanyaan apa yang terjadi sekarang, sementara visualisasi historis (line chart, bar chart dengan filter rentang waktu) menjawab pertanyaan bagaimana polanya dari waktu ke waktu. Praktik terbaik biasanya menggabungkan keduanya dalam satu dashboard: bagian atas untuk kondisi real-time, bagian bawah untuk grafik historis.

3.4. Threshold dan Alert

Threshold adalah ambang batas nilai yang menentukan status normal/tidak normal:

```
if (stok < 2) { status = "Kritis"; warna = "merah"; }
else if (stok < 5) { status = "Hampir Habis"; warna = "kuning"; }
else { status = "Normal"; warna = "hijau"; }
```

BAB 4 – Web Dashboard

4.1. Konsep Dasar Web Dashboard

Web dashboard adalah dashboard yang diakses melalui browser, dibangun menggunakan teknologi web (HTML, CSS, JavaScript) baik secara custom maupun melalui platform siap pakai. Berbeda dari website statis biasa, web dashboard IoT harus mampu menangani data yang terus

berubah secara efisien, inilah yang membedakan pendekatan pengembangannya dari website pada umumnya.

4.2. Komponen Utama Web Dashboard

- Frontend (tampilan) = Struktur (HTML), gaya (CSS), interaktivitas (JavaScript)
- Data fetching = Cara dashboard mengambil data terbaru dari server/database
- Rendering engine = Proses menampilkan data ke elemen visual dan memperbaruinya saat ada perubahan

4.3. Dua Cara Dashboard Mendapatkan Data Baru

Metode	Cara Kerja	Kelebihan	Kekurangan
Polling	Dashboard secara berkala mengirim request ke server (setInterval)	Sederhana diimplementasikan	Boros resource, delay tergantung interval
Push/Listener	Server otomatis mendorong data begitu ada perubahan	Efisien dan terasa lebih real-time	Setup lebih kompleks

1. Contoh Polling

```
setInterval(async () => {  
  const res = await fetch('/api/sensor/stok');  
  const data = await res.json();  
  document.getElementById('stok').innerText = data.nilai + ' kg';  
}, 2000);
```

2. Contoh Listener

```
firebase.database().ref('sensor/stok').on('value', (snapshot) => {  
  const data = snapshot.val();  
  document.getElementById('stok').innerText = data.nilai + ' kg';  
});
```

```
updateChart(data.timestamp, data.nilai);  
});
```

Pemilihan antara kedua metode ini cukup berpengaruh terhadap pengalaman pengguna dan efisiensi sistem. Polling lebih mudah dipahami pemula karena logikanya mirip "cek berkala", tetapi kurang efisien untuk skala besar karena tetap melakukan request meski tidak ada data baru. Listener/push lebih efisien dan lebih dekat dengan makna "real-time" yang sesungguhnya, karena dashboard hanya diperbarui saat benar-benar ada perubahan data.

4.4. Web Dashboard via Platform Siap Pakai (Node-RED Dashboard)

Selain dibangun manual dengan HTML/CSS/JS, web dashboard juga bisa dibuat melalui Node-RED Dashboard — modul tambahan Node-RED yang otomatis menghasilkan halaman web dari node UI (ui_gauge, ui_chart, ui_text) yang disusun secara visual (drag-and-drop), tanpa perlu menulis kode tampilan sama sekali. Pendekatan ini mempercepat pengembangan prototipe, meskipun kustomisasi visualnya lebih terbatas dibanding dashboard custom.

4.5. Struktur Umum Halaman Web Dashboard (Pendekatan Custom)

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">  
<div class="dashboard-grid">  
  <div class="card">Stok: <span id="stok">-</span> kg</div>  
  <div class="card">Status: <span id="status">-</span></div>  
  <canvas id="chartStok"></canvas>  
</div>
```

Dashboard-grid biasanya diatur dengan CSS Grid/Flexbox agar layout rapi, sementara elemen canvas digunakan oleh library grafik seperti Chart.js untuk merender visualisasi.

4.6. Update Otomatis Tanpa Refresh Manual

Kunci dari web dashboard IoT adalah kemampuan DOM manipulation atau mengubah isi halaman secara dinamis lewat JavaScript tanpa reload halaman

```
function updateChart(waktu, nilai) {  
  chart.data.labels.push(waktu);  
  chart.data.datasets[0].data.push(nilai);  
}
```

```
chart.update();  
}
```

BAB 5 - Mobile Monitoring

5.1. Dashboard yang Dapat Diakses via Smartphone

Dashboard IoT modern umumnya dirancang agar dapat diakses tidak hanya dari PC, tetapi juga dari perangkat mobile, mengingat pengguna sering membutuhkan pemantauan saat bergerak atau di luar ruangan.

5.2. Cara Membuat Dashboard Bisa Diakses via Mobile

5.2.1. Responsive Web Design

```
@media (max-width: 600px) {  
  .dashboard-grid { grid-template-columns: 1fr; }  
}
```

Progressive Web App (PWA), website yang bisa "diinstall" di HP seperti aplikasi, mendukung notifikasi push dan bekerja offline sebagian. Untuk kebutuhan tugas kuliah, PWA atau responsive web sudah cukup memadai dibanding harus membangun native app terpisah.

5.2.2. Mobile Monitoring via Firebase

Pendekatan umum: data sensor dikirim ke Firebase Realtime Database, lalu aplikasi/halaman mobile membaca data tersebut menggunakan listener:

```
firebase.database().ref('sensor/data').on('value', (snapshot) => {  
  const data = snapshot.val();  
  // update tampilan mobile  
});
```

Notifikasi bisa diimplementasikan melalui push notification (Firebase Cloud Messaging), Bot Telegram (relatif mudah untuk mahasiswa), atau email alert (paling sederhana, via SMTP).

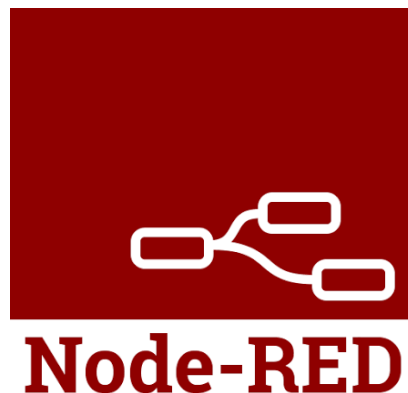
5.3. Perbedaan Monitoring PC vs Smartphone

Aspek	PC	Smartphone
Mobilitas	Rendah	Tinggi
Ukuran layar	Besar	Kecil, perlu tampilan ringkas
Notifikasi	Terbatas	Mendukung push notification
Kepraktisan cek cepat	Kurang praktis	Sangat praktis

Mobile monitoring pada dasarnya meningkatkan fleksibilitas karena pengguna tidak harus berada di depan komputer untuk mengetahui kondisi sistem. Lebih jauh, kombinasi mobile monitoring dengan notifikasi push mengubah paradigma monitoring dari pull-based (pengguna harus aktif membuka dashboard) menjadi push-based (sistem yang mendatangi pengguna saat ada kondisi penting) — hal ini sangat relevan untuk skenario kritis seperti kebocoran gas atau suhu mesin berlebih, di mana keterlambatan mengecek dashboard secara manual bisa berakibat fatal.

BAB 6 - Teknologi Pendukung Web & Mobile Dashboard

6.1. Node-RED



Node-RED adalah tool berbasis visual flow programming mahasiswa menyusun alur logika pemrosesan data dengan menghubungkan "node" (blok fungsi) secara drag-and-drop, tanpa perlu menulis banyak baris kode.

6.1.1. Komponen Dasar dalam Node-RED

Elemen	Fungsi
Node	Satu blok fungsi tunggal, seperti input, proses, atau output
Wire (Kabel)	Penghubung antar-node yang menentukan alur data

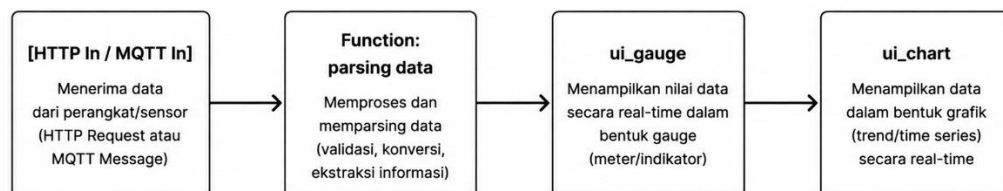
Flow	Kumpulan node yang saling terhubung dan membentuk satu alur kerja
Payload	Data yang mengalir dari satu node ke node berikutnya

6.1.2. Peran Node-RED untuk Web Dashboard

Node-RED menyediakan modul tambahan bernama Node-RED Dashboard (node-red-dashboard), yang secara otomatis menghasilkan halaman web dari node UI yang disusun di flow, tanpa perlu menulis HTML/CSS/JS manual. Node UI yang umum dipakai:

Node	Fungsi di Web Dashboard
ui_gauge	Menampilkan nilai dalam bentuk gauge, misalnya suhu atau level baterai
ui_chart	Menampilkan grafik tren data dari waktu ke waktu
ui_text	Menampilkan nilai dalam bentuk teks atau card
ui_switch / ui_button	Menyediakan elemen kontrol untuk mengirim perintah ke aktuator

Contoh alur sederhana:



Setiap kali ada data baru masuk ke flow, Node-RED Dashboard memperbarui tampilan secara otomatis melalui mekanisme push berbasis WebSocket, bukan polling sehingga web dashboard terasa real-time tanpa perlu refresh manual.

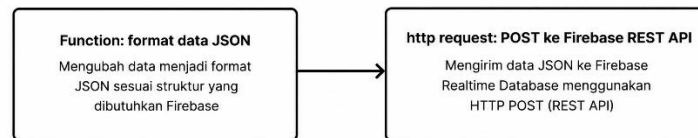
6.1.3. Peran Node-RED untuk Mobile Monitoring

Node-RED sendiri tidak secara langsung membuat aplikasi mobile, tetapi berperan sebagai jembatan pengolahan data sebelum data diteruskan ke layanan yang mendukung akses mobile (biasanya Firebase). Dalam alur ini, Node-RED bertugas:

- Menerima data mentah dari sensor/mikrokontroler

- Melakukan validasi dan transformasi (misalnya konversi satuan, pengecekan threshold)
- Meneruskan hasil olahan ke Firebase menggunakan node http request atau node Firebase khusus (node-red-contrib-firebase)

Contoh node output ke Firebase:



6.1.4. Kelebihan dan Kekurangan Node-RED

Kelebihan	Kekurangan
Tidak perlu banyak coding, cocok untuk pemula	Kustomisasi visual web terbatas dibandingkan HTML/CSS manual
Cepat membangun prototipe dashboard	Kurang cocok untuk logika pemrosesan yang sangat kompleks
Mendukung banyak protokol, seperti MQTT, HTTP, dan WebSocket	Perlu di-hosting sendiri (server/Raspberry Pi/komputer lokal) agar selalu aktif

6.2. Firebase



Firebase adalah platform backend dari Google yang menyediakan berbagai layanan cloud, dengan fitur utama yang relevan untuk dashboard IoT adalah database dengan sinkronisasi data secara real-time.

6.2.1. Dua Jenis Database Firebase

Jenis	Struktur Data	Kelebihan	Cocok Untuk
Realtime Database	JSON tree (bertingkat)	Sinkronisasi sangat cepat dan sederhana	Data sensor sederhana dengan update frekuensi tinggi
Firestore	Collection & document	Query lebih kompleks dan struktur data lebih terorganisir	Data dengan relasi atau filter yang lebih rumit

6.2.2. Peran Firebase untuk Web Dashboard

Firebase dapat digunakan sebagai sumber data untuk web dashboard custom (dibangun dengan HTML/CSS/JS), dengan cara web melakukan listen langsung ke perubahan data:

```
firebase.database().ref('sensor/data').on('value', (snapshot) => {  
  const data = snapshot.val();  
  document.getElementById('suhu').innerText = data.suhu + '°C';  
});
```

Dengan pendekatan ini, setiap kali ada data baru di Firebase (misalnya dikirim dari Node-RED atau langsung dari ESP32), tampilan web otomatis berubah tanpa perlu refresh halaman.

6.2.3. Peran Firebase untuk Mobile Monitoring

Firebase adalah pilihan populer untuk mobile monitoring karena:

- Menyediakan SDK resmi untuk Android, iOS, dan web mobile, sehingga mudah diintegrasikan ke aplikasi
- Mendukung listener real-time yang sama seperti di web, sehingga aplikasi mobile otomatis menerima update data

- Menyediakan Firebase Cloud Messaging (FCM) untuk push notification, misalnya mengirim peringatan saat suhu melewati threshold

Contoh struktur data yang tersimpan di Firebase untuk keperluan mobile:

```
{
  "sensor": {
    "suhu": 28.5,
    "kelembaban": 65,
    "status": "Normal",
    "timestamp": "2026-08-25T20:48:12"
  }
}
```

6.2.4. Kelebihan dan Kekurangan Firebase

Kelebihan	Kekurangan
Setup cepat dan dokumentasi lengkap	Struktur JSON tree dapat menjadi rumit untuk data kompleks, terutama pada Realtime Database
Sinkronisasi real-time otomatis ke banyak client sekaligus	Biaya dapat meningkat seiring jumlah pembacaan dan penulisan data di luar tier gratis
SDK siap pakai untuk web dan mobile	Bergantung pada layanan cloud dan tidak dapat di-self-host

6.3. ThingsBoard



ThingsBoard adalah platform IoT open-source yang menyediakan solusi all-in-one: database, dashboard builder, device management, rule engine, dan sistem alarm — semua dalam satu paket siap pakai.

6.2.1. Fitur Utama ThingsBoard

Fitur	Fungsi	Fitur
Device Management	Mengelola banyak perangkat IoT dalam satu platform	Device Management
Dashboard Builder	Menyusun widget seperti gauge, chart, map, dan tabel secara drag-and-drop	Dashboard Builder
Rule Engine	Membuat logika otomatis, misalnya mengirim alert jika suhu > 35°C	Rule Engine
Widget Library	Menyediakan kumpulan komponen visual siap pakai untuk berbagai kebutuhan	Widget Library

6.2.2. Peran ThingsBoard untuk Web Dashboard

ThingsBoard menyediakan dashboard builder berbasis web yang jauh lebih lengkap dibanding Node-RED Dashboard — mendukung banyak jenis widget (peta lokasi, tabel data kompleks, kontrol perangkat), serta tampilan yang bisa disesuaikan tanpa coding sama sekali. Web dashboard ThingsBoard diakses melalui portal bawaan platform ini, bukan file HTML terpisah.

6.2.3. Peran ThingsBoard untuk Mobile Monitoring

ThingsBoard mendukung akses mobile melalui:

- Aplikasi mobile resmi ThingsBoard, yang bisa menampilkan dashboard yang sudah dibuat di web portal
- Tampilan web yang responsif, sehingga dashboard ThingsBoard tetap bisa dibuka dari browser HP
- Notifikasi/alarm yang terintegrasi dengan rule engine, bisa dikirim ke email atau aplikasi

6.2.4. Perbandingan Cara Kerja ThingsBoard vs Node-RED+Firebase

Aspek	ThingsBoard (All-in-One)	Node-RED + Firebase (Kombinasi)
Setup awal	Perlu instalasi/hosting platform ThingsBoard	Setup terpisah: Node-RED sendiri dan Firebase sendiri
Dashboard	Dashboard builder bawaan, sangat lengkap	Node-RED Dashboard lebih sederhana
Mobile	Aplikasi mobile resmi tersedia	Perlu membangun sendiri menggunakan Firebase SDK
Fleksibilitas kustomisasi	Sedang, mengikuti widget yang tersedia	Tinggi, dapat disesuaikan secara bebas
Kompleksitas	Lebih tinggi pada tahap awal karena setup platform	Lebih modular, dapat dimulai dari konfigurasi sederhana

6.2.5. Kelebihan dan Kekurangan ThingsBoard

Kelebihan	Kekurangan
Solusi lengkap dalam satu platform	Kurva belajar lebih tinggi dibandingkan Node-RED
Dashboard builder sangat kaya fitur	Membutuhkan resource server lebih besar untuk hosting

Mendukung banyak perangkat sekaligus dan cocok untuk skala besar	Kurang fleksibel untuk logika pemrosesan data yang sangat spesifik
--	--

6.4. Perbandingan Ringkas Ketiga Tools

Tools	Peran Utama	Dukungan Web	Dukungan Mobile	Butuh Coding?
Node-RED	Pengolahan data & routing	Ya, melalui Node-RED Dashboard	Tidak langsung, dapat melalui Firebase	Minim (visual flow)
Firebase	Database & sinkronisasi real-time	Ya, melalui listener JavaScript	Ya, melalui SDK mobile	Ya (JavaScript)
ThingsBoard	Platform IoT lengkap	Ya, dashboard builder bawaan	Ya, aplikasi resmi	Minim (drag-and-drop)

BAB 7 – Real-Time Data

7.1. Pengertian Real-Time

Real-time dalam konteks dashboard IoT berkaitan dengan konsep data freshness seberapa segar data yang ditampilkan dibanding kondisi sebenarnya di lapangan. Semakin kecil selisih waktu antara kejadian fisik dan tampilan di dashboard, semakin real-time sistem tersebut dianggap.

7.2. Alur Proses

1. Pengiriman data sensor.

Sensor membaca nilai secara berkala, data dikemas dalam format JSON

2. Update database.

Data baru disimpan/menimpa nilai sebelumnya, sekaligus dicatat sebagai log historis

3. Sinkronisasi dashboard.

Via polling atau listener.

4. Interval pengiriman data.

Interval terlalu cepat boros bandwidth dan membebani server, sementara interval terlalu lambat berisiko membuat kondisi penting terlambat terdeteksi; umumnya interval 1–5 detik sudah cukup memadai

5. Latency.

Delay bisa terjadi di berbagai titik sepanjang alur sistem, dan setiap tahap menyumbang delay kecil yang terakumulasi menjadi total latency

7.3. Contoh

Sensor membaca:

10:00:01 → 10,0 kg

10:00:02 → 9,8 kg

10:00:03 → 9,7 kg

Dashboard ikut berubah otomatis tanpa refresh, karena terhubung ke database melalui listener real-time.

7.4. Faktor yang Memengaruhi Real-Time

Faktor	Dampak
Kecepatan internet	Memengaruhi delay pengiriman dan penerimaan data
Interval pembacaan sensor	Data terasa “lambat” diperbarui jika interval terlalu besar
Beban server	Respons menjadi lebih lambat jika terdapat banyak request
Jumlah device terhubung	Beban broker/database meningkat seiring bertambahnya perangkat

Penting dipahami bahwa real-time tidak selalu berarti tanpa delay sama sekali, ada kemungkinan delay karena sensor, jaringan, server, database, maupun proses visualisasi. Istilah yang lebih akurat secara teknis adalah near real-time, dan toleransi delay yang dapat diterima bergantung pada konteks aplikasi: monitoring stok gudang mungkin cukup dengan delay beberapa detik, sedangkan sistem kendali otomatis seperti rem otomatis membutuhkan delay yang jauh lebih kecil.

BAB 8 – STUDI KASUS : Dashboard Monitoring Suhu & Kelembaban Udara

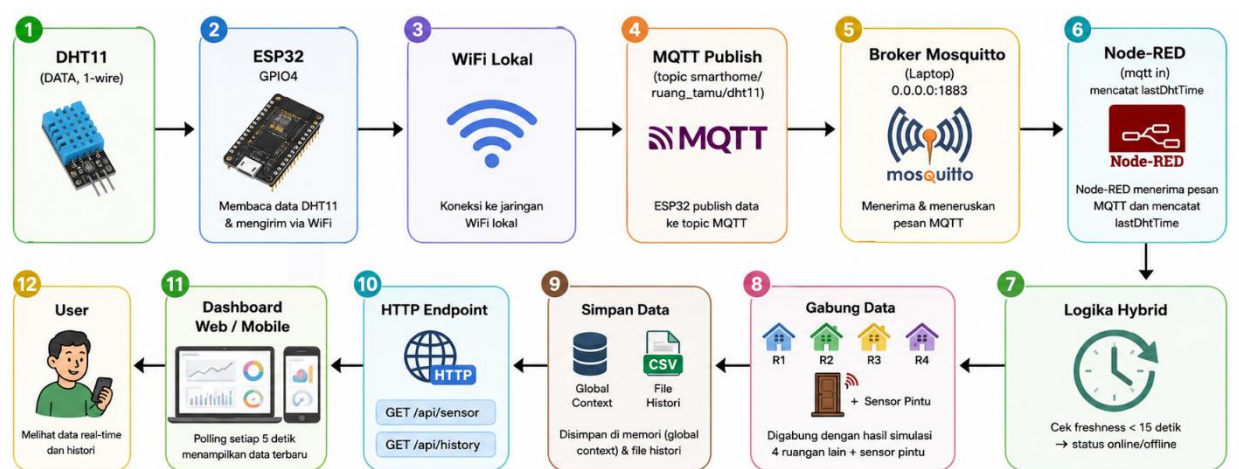
8.1. Gambaran Umum Studi Kasus

Studi kasus ini membangun dashboard IoT bergaya "Home Control" untuk memantau kondisi 5 ruangan sekaligus: 1 ruangan (Ruang Tamu) diukur menggunakan sensor fisik DHT11 yang terhubung ke ESP32, sedangkan 4 ruangan lainnya (Kamar 1, Kamar 2, Ruang Kerja, Dapur) disimulasikan menggunakan `Math.random()` di Node-RED. Pendekatan hybrid ini dipilih karena keterbatasan jumlah sensor fisik yang tersedia, namun tetap membuktikan alur data end-to-end yang sesungguhnya dari sensor asli.

Berbeda dari pendekatan dummy murni, arsitektur ini menambahkan lapisan MQTT (broker Mosquitto) sebagai penghubung antara ESP32 dan Node-RED, serta menerapkan mekanisme deteksi offline yang jujur: jika sensor DHT11 terputus, dashboard tidak menampilkan data dummy pengganti, melainkan status "Sensor Offline" dengan nilai kosong (null).

Repo: https://github.com/naufalm088/IOT- Kel3_Dashboard_Monitoring_Suhu_Kelembapan.git

8.2. Alur Sistem



Berdasarkan gambar di atas, sistem bekerja dengan membaca data suhu dan kelembaban menggunakan sensor DHT11 yang terhubung ke ESP32 melalui GPIO4. Data tersebut kemudian dikirim oleh ESP32 melalui jaringan WiFi lokal menggunakan protokol MQTT dengan topik `smarthome/ruang\_tamu/dht11`. Pesan MQTT diterima oleh broker Mosquitto yang berjalan pada laptop melalui port 1883, kemudian diteruskan ke Node-RED.

Pada Node-RED, data diterima melalui node `mqtt in` dan waktu penerimaan terakhir dicatat dalam variabel `lastDhtTime`. Sistem selanjutnya menerapkan logika hybrid untuk memeriksa kesegaran data. Apabila data terakhir diterima dalam waktu kurang dari 15 detik, sensor dianggap berstatus online, sedangkan jika melebihi batas tersebut sensor dianggap offline.

Data dari DHT11 kemudian digabungkan dengan data simulasi dari empat ruangan lainnya serta sensor pintu. Seluruh data disimpan pada `global context` Node-RED dan file histori. Data tersebut dapat diakses melalui endpoint HTTP `GET /api/sensor` untuk data sensor terkini dan `GET /api/history` untuk data histori. Dashboard Web/Mobile melakukan polling setiap 5 detik untuk mengambil dan menampilkan data terbaru kepada pengguna.

8.3. Komponen Hardware

No	Komponen	Jumlah	Fungsi
1	ESP32 DevKit	1 pcs	Mikrokontroler utama: membaca data sensor & mengirim via WiFi
2	Sensor DHT11 (modul 3-pin)	1 pcs	Mengukur suhu (0–50°C) dan kelembaban (20–90% RH) sekaligus
3	Kabel jumper female-to-female	3 pcs	Menghubungkan VCC, GND, DATA DHT11 langsung ke pin header ESP32
4	Kabel USB	1 pcs	Upload program & sumber daya ESP32
5	Laptop (Mosquitto + Node-RED)	1	Menjalankan broker MQTT (0.0.0.0:1883) dan server Node-RED (localhost:1880)

Penjelasan Komponen

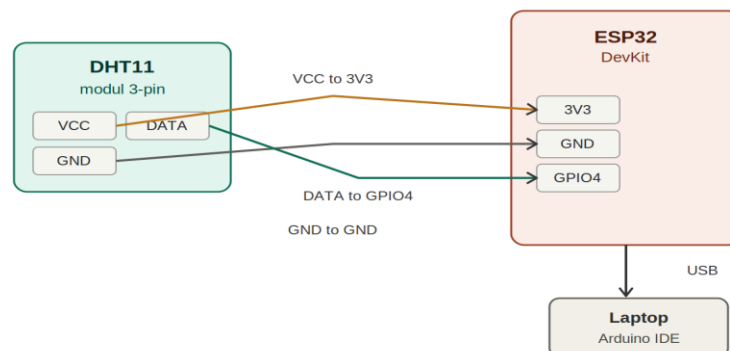
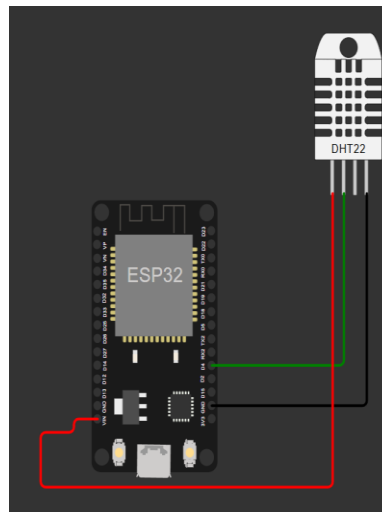
- ESP32 DevKit, mikrokontroler dengan modul WiFi terintegrasi, sehingga dapat langsung terhubung ke jaringan nirkabel tanpa perangkat tambahan. Dipilih karena proyek ini memerlukan pengiriman data secara real-time melalui internet/jaringan lokal.
- Sensor DHT11, sensor digital yang mampu membaca suhu (0–50°C) dan kelembaban (20–90% RH) sekaligus dalam satu komponen, berkomunikasi melalui protokol 1-wire. Modul

3-pin yang digunakan sudah memiliki resistor pull-up internal, sehingga tidak memerlukan komponen tambahan.

- Kabel jumper female-to-female, dipilih karena kedua ujung komponen (pin DHT11 dan pin header ESP32) sama-sama berupa pin fisik yang menonjol, sehingga dapat terhubung langsung tanpa breadboard sebagai perantara.
- Kabel USB, selain untuk mengunggah program dari Arduino IDE ke ESP32, kabel ini juga menjadi sumber daya sementara selama pengujian dilakukan melalui laptop.

Modul DHT11 3-pin yang dipakai sudah memiliki resistor pull-up internal, sehingga tidak memerlukan resistor tambahan. Karena kedua ujung komponen (pin DHT11 dan pin header ESP32) sama-sama pin fisik yang menonjol, sambungan dilakukan langsung (point-to-point) tanpa breadboard, menggunakan 3 kabel jumper female-to-female.

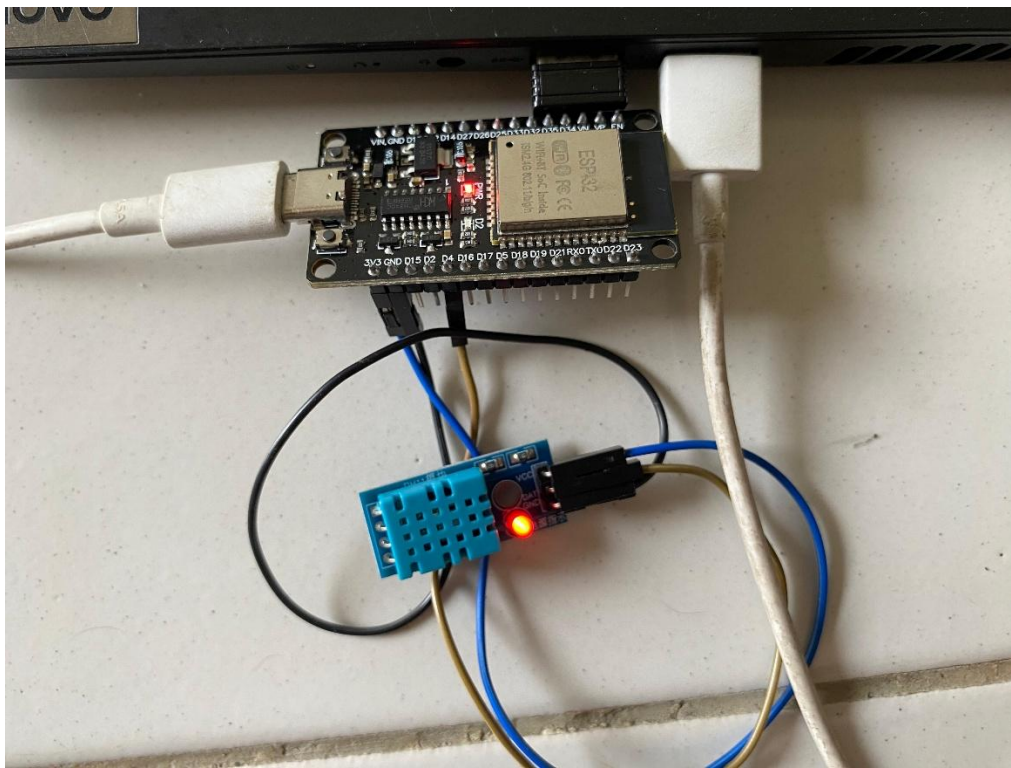
8.4. Wiring Diagram



Wiring langsung dengan 3 kabel jumper female-to-female, tanpa breadboard

Pin DHT11	Pin ESP32	Fungsi	Keterangan
VCC	3V3	Power 3.3V	Jangan pakai 5V, logic ESP32 adalah 3.3V
GND	GND	Ground	Referensi tegangan nol bersama
DATA	GPIO4 (D4)	Jalur data 1-wire	Sesuai #define DHTPIN 4 pada program

8.5. Proses Pemasangan Bertahap



1. Siapkan komponen ESP32 dan DHT11.
2. Hubungkan VCC: Jumper merah dari ESP32 3V3 ke rel + breadboard → ke DHT11 VCC (tengah).
3. Hubungkan GND: Jumper hitam dari ESP32 GND ke rel - → ke DHT11 GND (kanan).
4. Hubungkan DATA: Jumper kuning dari ESP32 D4 ke jalur DHT11 DATA (kiri) — tanpa resistor.
5. Hubungkan USB: Kabel CH340 ke COM7 (cek Device Manager > Ports), baud 115200.
6. Verifikasi fisik: LED ESP32 menyala, tidak ada jumper longgar, Serial Monitor siap.

8.6. Kode Program

Program ESP32 ditulis menggunakan Arduino IDE dan diunggah ke board. Berbeda dari versi dummy, bagian ini membaca sensor fisik secara langsung (bukan disimulasikan), sehingga nilai yang dikirim ke broker MQTT adalah hasil pengukuran nyata dari DHT11. Berikut bagian-bagian penting dari keseluruhan program

8.6.1. Library yang Digunakan

```
#include <WiFi.h>          // koneksi WiFi bawaan ESP32
#include <PubSubClient.h> // komunikasi protokol MQTT
#include "DHT.h"           // driver sensor DHT11/DHT22
// Library tambahan yang perlu diinstal melalui Library Manager:
// - DHT sensor library by Adafruit (versi 1.4.4)
// - Adafruit Unified Sensor (dependency dari DHT library)
// - PubSubClient by Nick O'Leary (versi 2.8)
```

Library yang digunakan adalah:

- WiFi.h, library bawaan ESP32 yang digunakan untuk menghubungkan mikrokontroler ke jaringan WiFi lokal, sehingga ESP32 dapat mengirim data melalui internet/jaringan tanpa perangkat tambahan.
- PubSubClient.h (by Nick O'Leary, v2.8), library yang menangani komunikasi protokol MQTT, digunakan untuk melakukan publish data suhu dan kelembaban ke broker Mosquitto pada topic smarthome/ruang_tamu/dht11.
- DHT.h (DHT sensor library by Adafruit, v1.4.4, bersama dependency-nya Adafruit Unified Sensor), library yang menangani komunikasi 1-wire dengan sensor DHT11, digunakan untuk memanggil fungsi dht.readTemperature() dan dht.readHumidity().

8.6.2. Inisialisasi Sensor

```
#define DHTPIN 4    // pin data DHT11 terhubung ke GPIO4
#define DHTTYPE DHT11 // tipe sensor yang digunakan

DHT dht(DHTPIN, DHTTYPE); // objek sensor DHT
WiFiClient espClient;
```

```
PubSubClient client(espClient); // objek client MQTT
```

```
void setup() {  
  Serial.begin(115200);  
  dht.begin(); // memulai komunikasi dengan sensor  
  ...  
}
```

8.6.3. Pengaturan Pin dan Koneksi

```
// ----- WIFI -----  
const char* ssid = "Kos ijo";  
const char* password = "Aslan199";  
  
// ----- MQTT BROKER (IP laptop) -----  
const char* mqtt_server = "192.168.1.13";  
const int mqtt_port = 1883;  
const char* topic_data = "smarthome/ruang_tamu/dht11";  
  
// Menghubungkan ke WiFi pada fungsi setup():  
WiFi.begin(ssid, password);  
while (WiFi.status() != WL_CONNECTED) {  
  delay(500);  
  Serial.print(".");  
}  
Serial.printf("WiFi terhubung! IP: %s\n", WiFi.localIP().toString().c_str());  
  
// Menghubungkan ke broker MQTT:  
client.setServer(mqtt_server, mqtt_port);  
reconnectMQTT();
```

8.6.4. Proses Pembacaan Sensor

```
float temperature = dht.readTemperature(); // membaca suhu (°C)
float humidity = dht.readHumidity();      // membaca kelembaban (%)

// Validasi hasil pembacaan.
// Jika sensor gagal membaca (misal kabel longgar/terlepas),
// sistem akan mempublikasikan status "offline", bukan data dummy.
if (isnan(temperature) || isnan(humidity)) {
    Serial.println("[DHT11] Gagal membaca sensor! Cek wiring.");
    client.publish("smarthome/ruang_tamu/status", "offline", true);
    return;
}
client.publish("smarthome/ruang_tamu/status", "online", true);
```

8.6.5. Pengiriman Data

```
const unsigned long intervalMs = 5000; // kirim data tiap 5 detik
unsigned long lastPublish = 0;

void loop() {
    if (!client.connected()) { reconnectMQTT(); }
    client.loop();

    unsigned long now = millis();
    if (now - lastPublish >= intervalMs) {
        lastPublish = now;

        // ... pembacaan sensor (lihat 8.5.4) ...

        // Membentuk data dalam format JSON
```

```

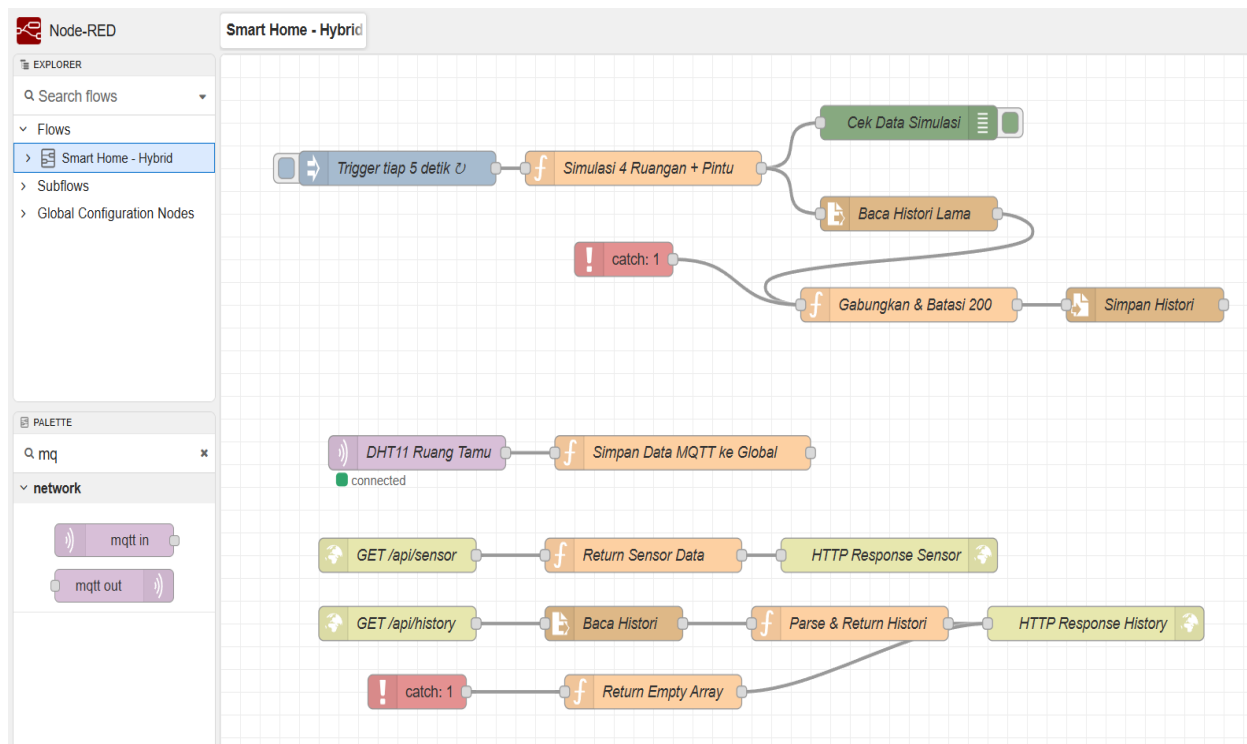
char payload[80];
snprintf(payload, sizeof(payload),
    "{\"temperature\":%.1f,\"humidity\":%.1f}",
    temperature, humidity);

// Mengirim (publish) data ke broker MQTT
client.publish(topic_data, payload);
Serial.printf("[MQTT] Publish -> %s : %s\n", topic_data, payload);
}
}

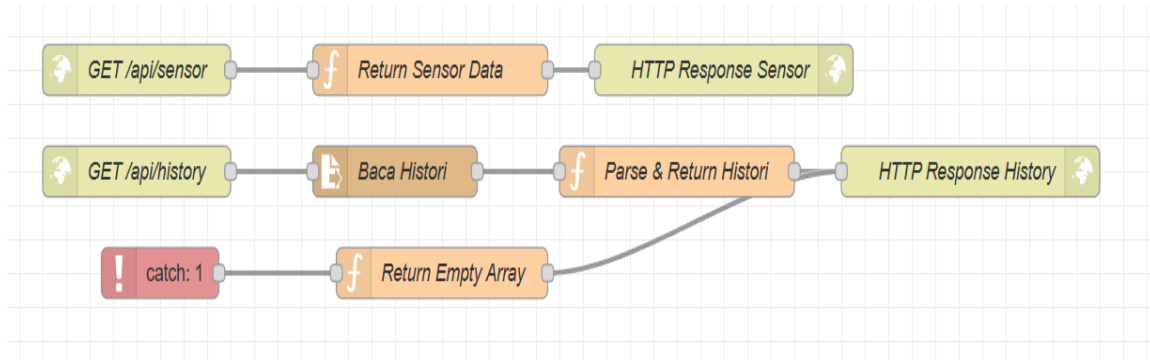
```

8.6.6. Node-RED: Logika Hybrid (Real + Simulasi)

Setelah data dari ESP32 sampai di broker MQTT, Node-RED bertindak sebagai subscriber sekaligus penggabung data real dan simulasi. Jika data dari topic MQTT ruang tamu tidak diterima dalam 15 detik terakhir, ruangan tersebut otomatis dianggap offline, bukan diisi angka acak pengganti.



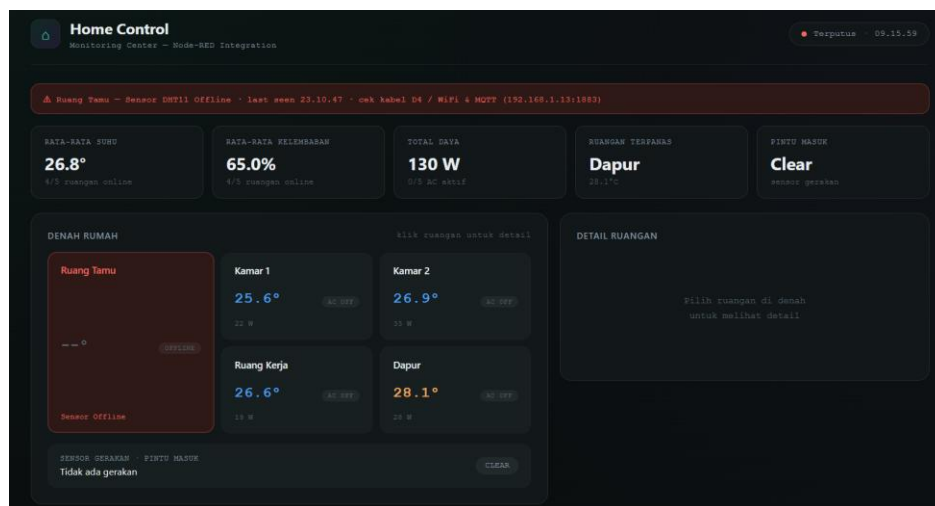
8.6.7. Node-RED Sebagai API

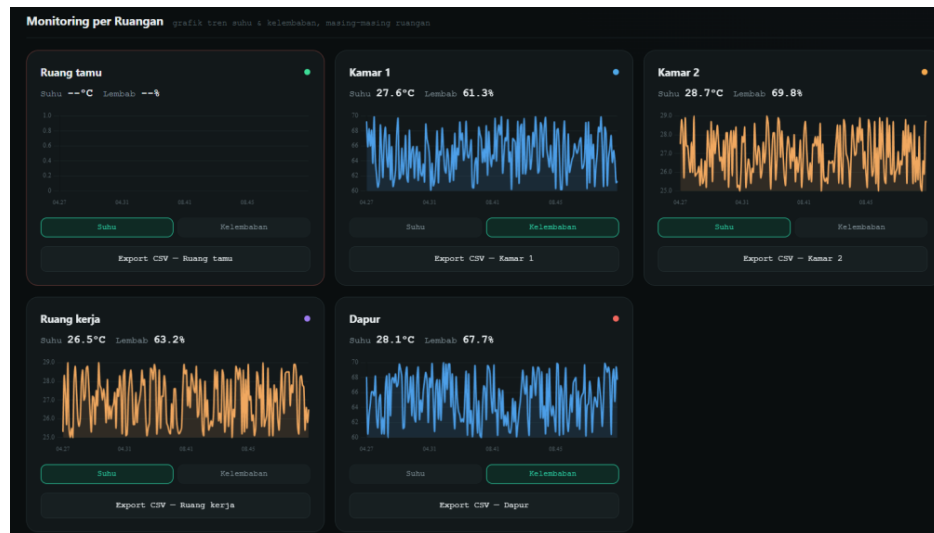
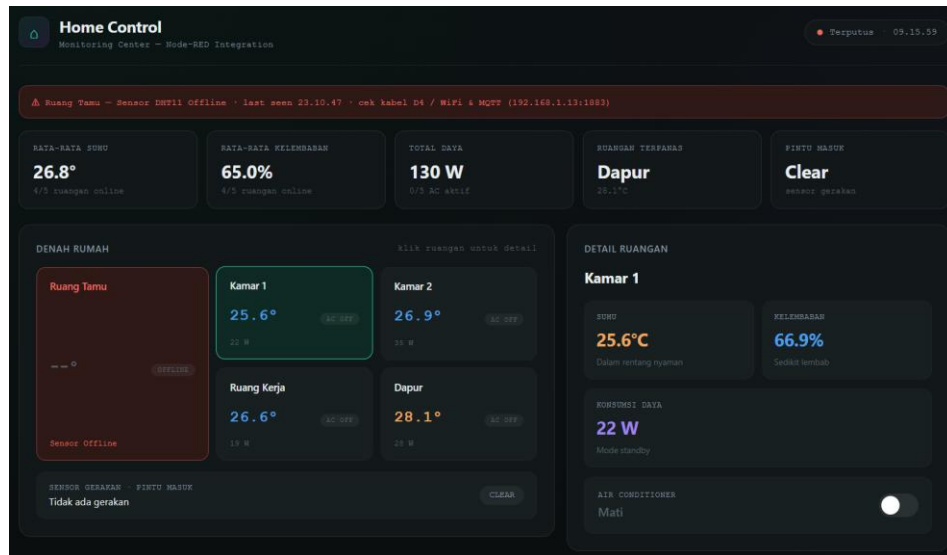


Tiga node dikombinasikan untuk menyediakan endpoint:

- HTTP In untuk mendefinisikan endpoint GET /api/sensor (dan GET /api/history)
- Function untuk mengambil data terbaru dari `global.get('latestSensorData')` atau membaca file histori
- HTTP Response untuk mengirim data sebagai JSON ke pemanggil

8.6.8. Dashboard Custom "Home Control" (HTML/CSS/JS)





Dashboard dibangun manual untuk kontrol tampilan penuh, dengan fitur:

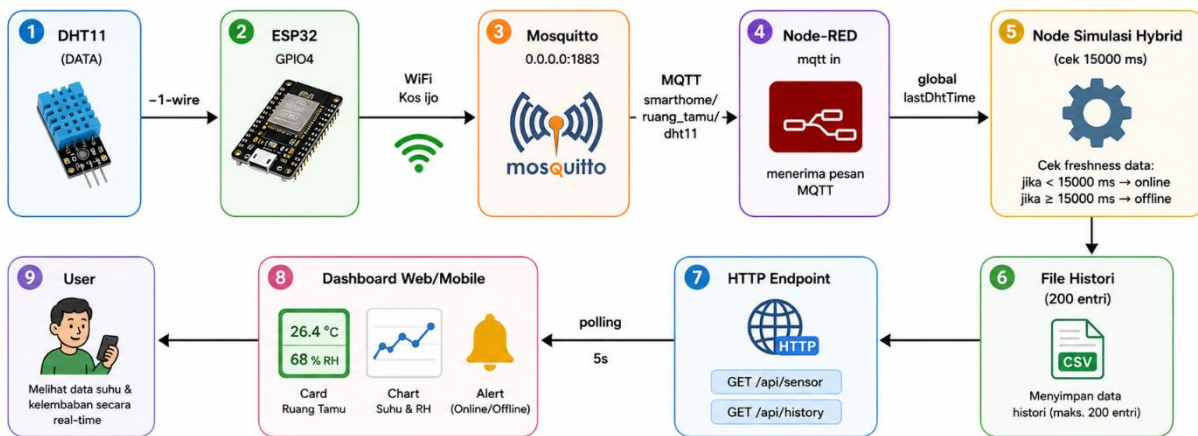
- 5 tile ringkasan: rata-rata suhu, rata-rata kelembaban, total daya(simulasi), ruangan terpanas, status pintu masuk
- Denah rumah (floorplan) interaktif — klik ruangan untuk detail
- Panel "Detail Ruangan" menampilkan suhu, kelembaban, konsumsi daya, dan toggle kontrol AC (simulasi)
- Grafik tren per ruangan dengan toggle Suhu/Kelembaban + tombol Export CSV per ruangan
- Panel "Analitik Sensor Pintu": total deteksi, jam tersibuk, dan distribusi gerakan per jam (bar chart) + export CSV log & rekam

Saat sensor DHT11 offline, tile Ruang Tamu di floorplan berubah warna merah dengan label "Sensor Offline" dan grafik menunjukkan gap (bukan garis dummy yang menyambung).

8.6.9. Polling Data

Dashboard mengambil data terbaru dengan `setInterval`, polling ke endpoint `/api/sensor` setiap 5 detik dan `/api/history` setiap 15 detik.

8.7. Proses Sampai Menghasilkan Data



1. Sensor (DHT11), mengukur suhu dan kelembaban udara di sekitar sensor secara langsung, menghasilkan nilai analog yang dikonversi menjadi data digital oleh sensor itu sendiri.
2. ESP32, membaca data digital dari DHT11 melalui pin GPIO4 menggunakan protokol 1-wire, memanggil fungsi `dht.readTemperature()` dan `dht.readHumidity()`.
3. Program (Arduino Sketch), mengemas data suhu dan kelembaban ke dalam format JSON, kemudian memvalidasi apakah pembacaan berhasil sebelum dikirim.
4. WiFi, ESP32 mengirimkan data melalui jaringan WiFi lokal menuju broker MQTT yang berjalan di laptop (alamat IP 192.168.1.13, port 1883).
5. MQTT Broker (Mosquitto), menerima data yang dipublikasikan (publish) oleh ESP32 pada topik `smarthome/ruang_tamu/dht11`, kemudian meneruskannya ke pelanggan (subscriber) yang berlangganan topik tersebut.
6. Node-RED, bertindak sebagai subscriber MQTT, menerima data, mencatat waktu penerimaan terakhir, memvalidasi status online/offline sensor, kemudian menyimpannya

sekaligus menyediakan endpoint HTTP (GET /api/sensor) agar data dapat diakses oleh aplikasi lain.

7. Dashboard, melakukan pengambilan data (polling) ke endpoint Node-RED setiap 5 detik menggunakan JavaScript, lalu menampilkannya secara visual dalam bentuk angka, indikator status, dan grafik tren kepada pengguna.

Dengan alur ini, pengguna dapat melihat kondisi suhu dan kelembaban ruangan secara real-time melalui browser, tanpa perlu mengakses ESP32 maupun broker MQTT secara langsung. Sistem juga dirancang untuk memberi peringatan yang jujur ketika sensor terputus dari rangkaian dashboard akan menampilkan status "Sensor Offline" alih-alih menampilkan data yang menyesatkan.

8.8. Hasil Pengujian

8.8.1. Serial Monitor (115200, ESP32)

```
[MQTT] Publish -> smarthome/ruang_tamu/dht11 : {"temperature":32.8,"humidity":62.0}
[MQTT] Publish -> smarthome/ruang_tamu/dht11 : {"temperature":32.8,"humidity":61.0}
[MQTT] Publish -> smarthome/ruang_tamu/dht11 : {"temperature":32.8,"humidity":61.0}
.....
WiFi terhubung! IP: 192.168.1.18
Menghubungkan MQTT 192.168.1.13:1883 ... Terhubung!
Topic: smarthome/ruang_tamu/dht11
[MQTT] Publish -> smarthome/ruang_tamu/dht11 : {"temperature":32.8,"humidity":62.0}
[MQTT] Publish -> smarthome/ruang_tamu/dht11 : {"temperature":32.8,"humidity":62.0}
[MQTT] Publish -> smarthome/ruang_tamu/dht11 : {"temperature":32.8,"humidity":61.0}
[MQTT] Publish -> smarthome/ruang_tamu/dht11 : {"temperature":32.8,"humidity":62.0}
[MQTT] Publish -> smarthome/ruang_tamu/dht11 : {"temperature":32.8,"humidity":61.0}
[MQTT] Publish -> smarthome/ruang_tamu/dht11 : {"temperature":32.8,"humidity":62.0}
```

8.8.2. Verifikasi via mosquito_sub

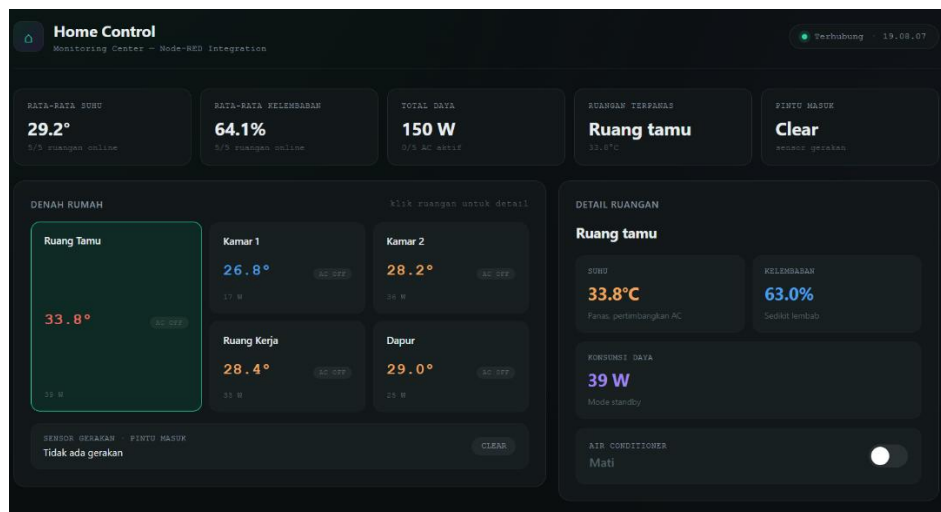
[illegible]

8.8.3. Node-RED Debug

```
rooms: {
  ruang_tamu: rtStatus === 'online' ? {
    temperature: dataBaru.rooms.ruang_tamu.temperature,
    humidity: dataBaru.rooms.ruang_tamu.humidity
  } : null,
  kamar_1: {
    temperature: dataBaru.rooms.kamar_1.temperature,
    humidity: dataBaru.rooms.kamar_1.humidity
  },
  kamar_2: {
    temperature: dataBaru.rooms.kamar_2.temperature,
    humidity: dataBaru.rooms.kamar_2.humidity
  },
  ruang_kerja: {
    temperature: dataBaru.rooms.ruang_kerja.temperature,
    humidity: dataBaru.rooms.ruang_kerja.humidity
  },
  dapur: {
    temperature: dataBaru.rooms.dapur.temperature,
    humidity: dataBaru.rooms.dapur.humidity
  }
}
```

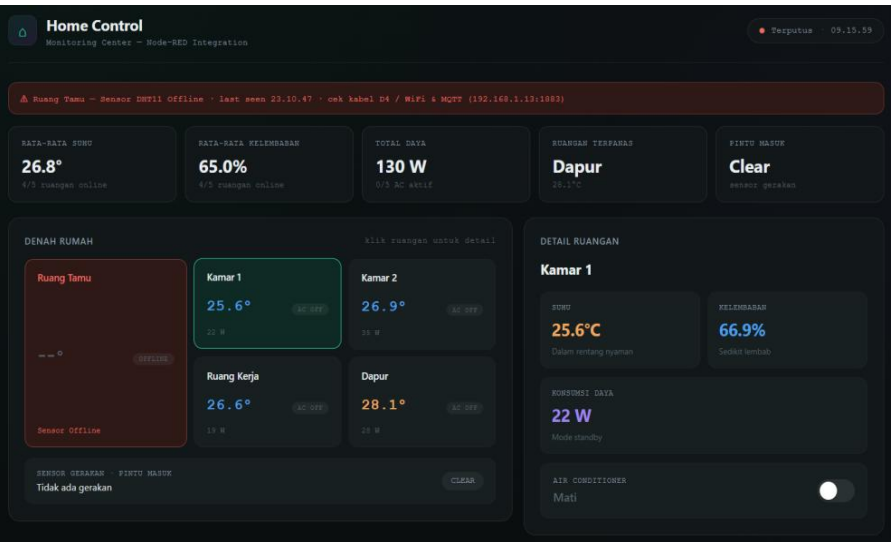
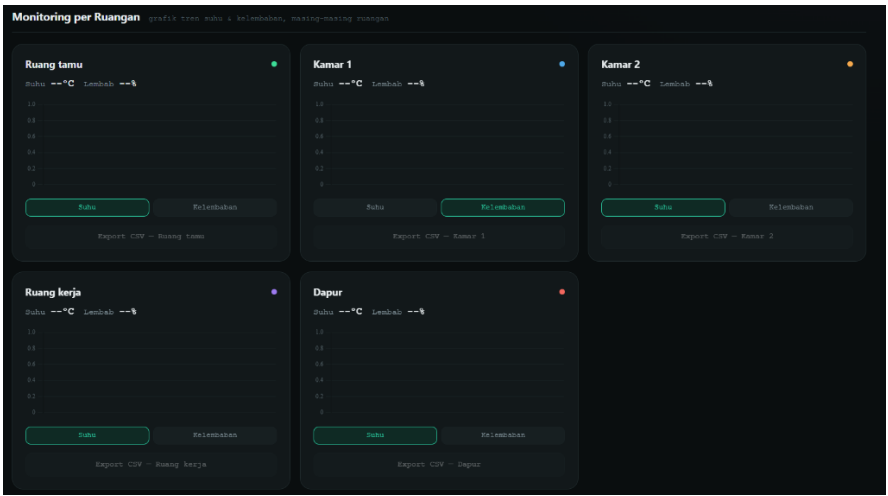
8.8.4. Dashboard Kondisi Normal vs Offline

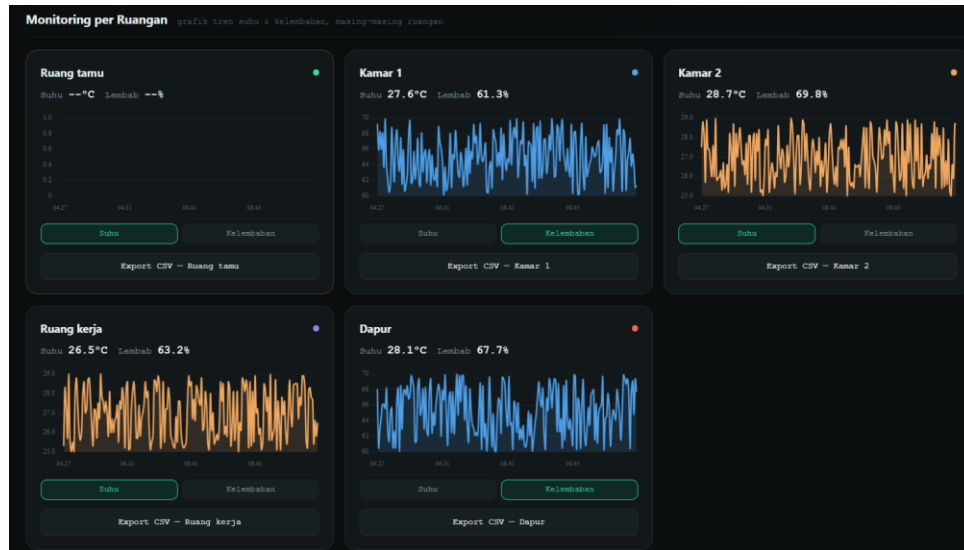
a. Kondisi Online





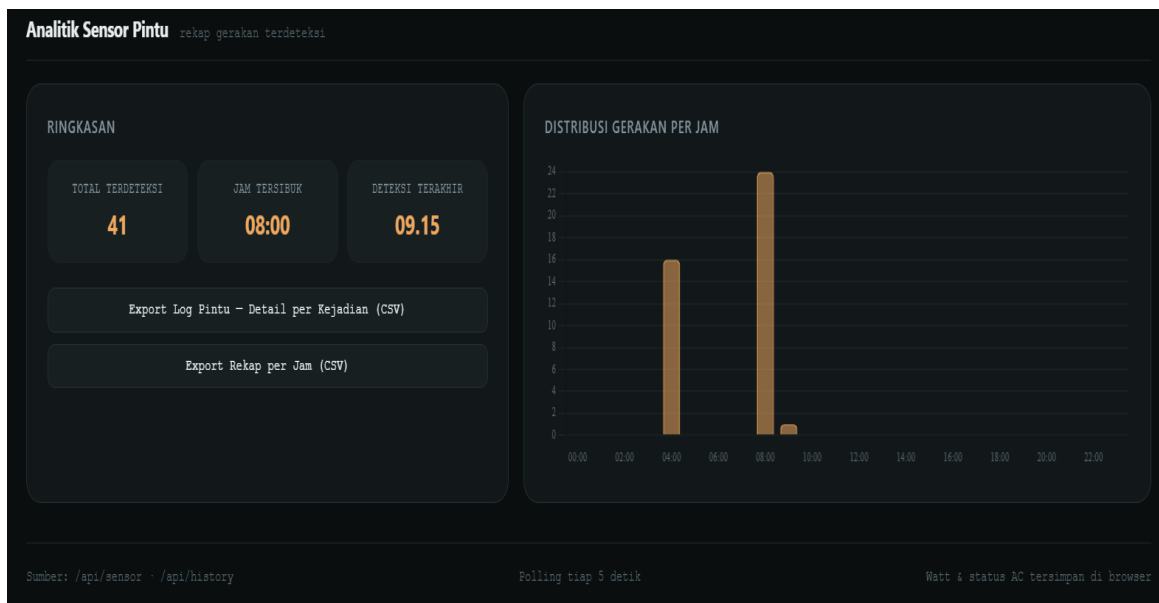
b. Kondisi Offline





Hasil pengujian menunjukkan bahwa sistem berhasil membaca data sensor secara akurat, mengirimkannya melalui MQTT tanpa kehilangan data, serta menampilkannya secara real-time pada dashboard dengan jeda pembaruan sekitar 5 detik sesuai interval pengiriman dari ESP32.

8.8.5. Histori & Analitik Sensor Pintu



Tanggal & Waktu,Suhu (Â°C),Kelembaban (%)						
A	B	C	D	E	F	G
1	Tanggal & Waktu,Suhu (Â°C),Kelembaban (%)					
2	01/09/2026, 19.03.50,34.2,64.0					
3	01/09/2026, 19.03.55,34.2,63.0					
4	01/09/2026, 19.04.00,34.2,63.0					
5	01/09/2026, 19.04.05,34.2,64.0					
6	01/09/2026, 19.04.10,34.2,63.0					
7	01/09/2026, 19.04.15,34.2,63.0					
8	01/09/2026, 19.04.20,34.2,63.0					
9	01/09/2026, 19.04.26,34.2,64.0					
10	01/09/2026, 19.04.31,34.2,63.0					
11	01/09/2026, 19.04.36,33.9,63.0					
12	01/09/2026, 19.04.41,33.8,63.0					
13	01/09/2026, 19.04.46,33.8,63.0					
14	01/09/2026, 19.04.51,33.8,63.0					
15	01/09/2026, 19.04.56,33.8,63.0					
16	01/09/2026, 19.05.01,33.8,63.0					
17	01/09/2026, 19.05.06,33.8,63.0					
18	01/09/2026, 19.05.11,33.8,64.0					
19	01/09/2026, 19.05.16,33.8,63.0					
20	01/09/2026, 19.05.21,33.8,64.0					
21	01/09/2026, 19.05.26,33.8,63.0					
22	01/09/2026, 19.05.31,33.8,63.0					
23	01/09/2026, 19.05.36,33.8,63.0					
24	01/09/2026, 19.05.41,33.8,63.0					
25	01/09/2026, 19.05.51,33.8,64.0					
26	01/09/2026, 19.05.56,33.8,64.0					
27	01/09/2026, 19.06.01,33.8,63.0					
28	01/09/2026, 19.06.06,33.8,63.0					

Fitur analitik sensor pintu menampilkan total deteksi gerakan, jam tersibuk, dan distribusi gerakan per jam dalam bentuk bar chart, lengkap dengan export CSV log per kejadian dan rekap per jam.

8.9. Kendala

Saat dashboard (dibuka sebagai file HTML lokal) mencoba memanggil API Node-RED, browser sempat memblokir request tersebut karena kebijakan CORS (Cross-Origin Resource Sharing) browser menganggap dashboard dan API berasal dari origin/sumber yang berbeda.

Solusi: mengaktifkan httpNodeCors di file konfigurasi settings.js Node-RED:Mendukung Mobile Monitoring Tanpa Aplikasi Terpisah

```

JS settings.js X
C: > Users > Naufal Mirza Aldilla > .node-red > JS settings.js > [?] <unknown>
23  module.exports = {
200  httpNodeCors: {
201    origin: "*",
202    methods: "GET,PUT,POST,DELETE",

```

Konfigurasi ini memberi tahu Node-RED untuk mengizinkan request dari origin manapun (origin: "*"), sehingga dashboard yang dibuka secara lokal tetap bisa mengakses endpoint API tanpa diblokir browser. Kendala ini adalah contoh nyata bahwa membangun dashboard bukan sekadar soal desain visual, tapi juga perlu memahami aturan keamanan bawaan browser saat menghubungkan komponen-komponen yang berbeda sumber.

8.10. Mendukung Mobile Monitoring Tanpa Aplikasi Terpisah

Karena dashboard dibangun dengan HTML/CSS yang responsif, tampilan yang sama otomatis bisa dibuka dari smartphone., cukup mengakses alamat IP lokal komputer (bukan localhost) selama perangkat berada di jaringan Wi-Fi yang sama, tanpa perlu membangun aplikasi mobile terpisah.



Ini adalah penerapan langsung dari konsep Responsive Web Design, satu basis kode dashboard melayani dua kanal akses (laptop dan HP) sekaligus, berbeda dari pendekatan yang memisahkan web dashboard dan mobile monitoring lewat dua teknologi berbeda (misalnya Node-RED Dashboard untuk web, Firebase untuk mobile).

8.11. Tools yang Digunakan

No	Tools	Fungsi
1	ESP32 DevKit	Membaca sensor & mengirim data via WiFi/MQTT
2	Sensor DHT11	Mengukur suhu & kelembaban ruang tamu (real)
3	Kabel jumper female-to-female	Menghubungkan DHT11 ke ESP32 tanpa breadboard
4	Mosquitto (MQTT Broker)	Menerima publish dari ESP32, meneruskan ke Node-RED
5	Node-RED	Subscriber MQTT, logika hybrid, endpoint API
6	Node-RED Function/Inject Node	Simulasi 4 ruangan lain & sensor pintu
7	Node-RED HTTP In/Response	Endpoint API (GET /api/sensor, GET /api/history)
8	HTML/CSS/JavaScript	Dashboard custom " Home Control "
9	Browser (Chrome)	Menguji API, dashboard, & simulasi tampilan mobile

8.12. Perbandingan dengan Alternatif Lain (Firebase & ThingsBoard)

Aspek	Node-RED + Dashboard Custom (Dipakai)	Firebase	ThingsBoard
Jenis tools	Middleware/orchestrator + frontend manual	Backend-as-a- Service umum (bukan khusus IoT)	Platform IoT siap pakai
Dashboard	Dibuat sendiri, desain bebas	Tidak ada bawaan, harus dibangun sendiri	Sudah tersedia (drag-and-drop widget)
Kompleksitas setup	Sedang, cukup Node- RED + kode frontend	Sedang, perlu kredensial, SDK, dan akun cloud	Sedang-tinggi, perlu provisioning device dan rule chain

Kontrol tampilan	Penuh , karena dibuat sendiri	Tergantung developer, tidak ada template bawaan	Terbatas pada widget yang disediakan
Cocok untuk	Prototype cepat dan proof of concept	Proyek yang membutuhkan cloud database & aplikasi mobile native	Proyek skala besar dengan banyak device
Alasan tidak dipakai penuh	-	Tidak dibutuhkan cloud storage untuk prototype ini	Dashboard bawaan dirasa berlebihan untuk skala prototype

Kesimpulan pemilihan tools: Node-RED + dashboard custom dipilih karena paling ringkas untuk membangun prototipe sekaligus memberi kontrol penuh atas tampilan. Firebase dan ThingsBoard tetap relevan sebagai opsi pengembangan lanjutan, Firebase jika project butuh penyimpanan cloud jangka panjang atau notifikasi push, dan ThingsBoard jika project berkembang ke skala banyak perangkat dengan kebutuhan device management yang lebih formal.

8.13. Analisis Studi Kasus

Studi kasus ini menunjukkan bahwa pendekatan hybrid yaitu dengan menggabungkan 1 sensor fisik dengan beberapa ruangan simulasi adalah strategi praktis ketika jumlah sensor fisik terbatas, tanpa mengorbankan esensi pembuktian alur data end-to-end (Sensor → ESP32 → MQTT → Node-RED → Dashboard).

Penambahan lapisan MQTT (Mosquitto) dibanding pendekatan HTTP polling murni pada studi kasus sebelumnya juga menunjukkan pola komunikasi publish-subscribe yang lebih dekat dengan praktik IoT industri, sekaligus tetap mempertahankan polling di sisi dashboard karena kesederhanaan implementasinya untuk skala prototipe.

Kendala CORS yang muncul kembali menegaskan bahwa membangun dashboard IoT tidak hanya soal desain visual, tapi juga memahami keamanan browser saat komponen sistem berasal dari origin berbeda.

Terakhir, keputusan menggunakan satu basis kode responsif untuk web dan mobile (dibanding pendekatan bercabang seperti Node-RED Dashboard untuk web + Firebase untuk mobile) menunjukkan bahwa kebutuhan mobile monitoring tidak selalu memerlukan teknologi tambahan, CSS responsif saja dikombinasikan dengan akses lewat jaringan lokal yang sama, sudah cukup untuk skala prototipe.

Link Presentasi : <https://canva.link/kvqn2638cm1sacv>